Design and Development of a Single-Page CanSat Ground Control Software (GCS) for Real-Time Telemetry Monitoring and Mission Operations

INDIA SPACE LAB

Project Report

17th June to 31st July 2026

Student Name: SRINJOY SIL
Institute Name: MCKV INSTITUTE OF ENGINEERING
Institute Roll no.: 11600223119
Enrollment no.: ISL-522235

Design and Development of a Single-Page CanSat Ground Control Software (GCS) for Real-Time Telemetry Monitoring and Mission Operations

CONTENTS

CHAPTERS	TITLE	PAGE NO.
Chapter 1	Introduction & System Architecture	1-4
Chapter 2	UI/UX Interface Layout	5-7
Chapter 3	Low-Latency Telemetry Parsing & Ingestion	8-10
Chapter 4	Real-Time Diagnostic & 4-Digit Error Code System	11-13
Chapter 5	Real-Time Spatial & Avionics Visualization Frameworks	14-16
Chapter 6	Mission Command Uplink & Data Management	17-19
Chapter 7	Testing Strategy, Mission Simulation & Conclusion	20-22

CHAPTER 1: INTRODUCTION & SYSTEM ARCHITECTURE

1.1 Project Overview and Ground Station Objectives

In aerospace engineering and satellite mission workflows, the Ground Control Station (GCS) serves as the centralized commanding nervous system. It bridges physical flight components with human operators on Earth. During a CanSat suborbital simulation or operational atmospheric descent, raw sensory parameters must be captured, parsed, and translated into high-fidelity actionable insights without structural processing delay.

The primary objective of this project is the end-to-end design and structural programming of a professional, low-latency, Single-Page Ground Control Software dashboard tailored to the *India Space Lab (ISL)* mission criteria. Utilizing modern browser-native execution pipes, this system eliminates bloated intermediate database stacks or back-end server hops. This ensures direct hardware-to-UI data mapping. The interface provides real-time situational awareness by displaying concurrent time-series plots, geographic coordinate tracks, active 3D roll-pitch-yaw attitude orientations, and multi-tiered fault detection arrays within a single window.

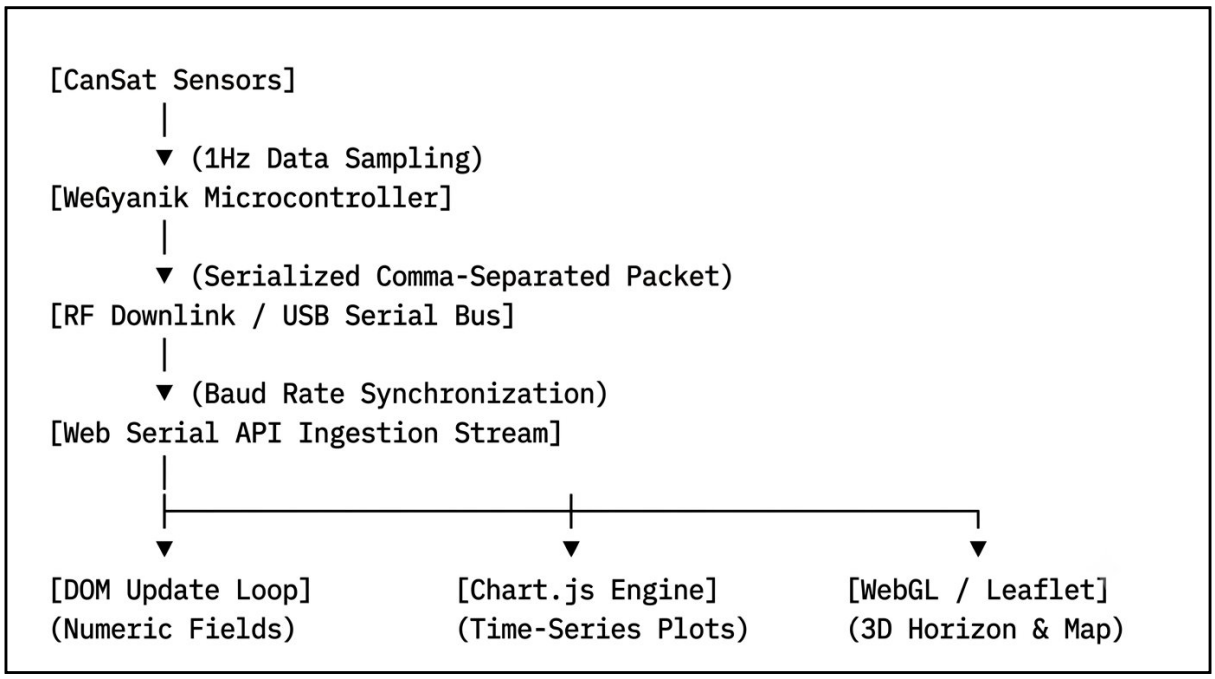
Beyond raw telemetry ingestion, the system incorporates dedicated command uplink channels and automated validation protocols to guarantee true closed-loop mission control. Integrated interactive buttons allow operators to dispatch critical, real-time override signals—including manual payload separation and emergency parachute deployment commands—directly back to the *WeGyanik Kit* microcontroller. Simultaneously, the architecture integrates a browser-native data management layer leveraging the Blob API. This layer continuously caches incoming flight history, enabling seamless session synchronization, instantaneous CSV log exports, and high-resolution chart snapshots directly from the local machine without requiring cloud persistence. By combining this localized data governance framework with programmatic error handling and hardware-accelerated WebGL visuals, this GCS layout establishes a reliable, portable, and industry-standard dashboard built specifically to withstand simulated flight failure states.

1.2 Comprehensive System Architecture and Data Flow Pipeline

The communication path of the CanSat mission system forms a reliable pipeline from physical atmospheric data collection to visual browser components:

1. Data Acquisition and Encoding Layer: The secondary payload and container arrays collect physical metrics (barometric pressure, internal/external temperatures, multi-axis accelerations, GPS coordinates, and power rails) at an execution frequency of ≥ 1Hz. The integrated *WeGyanik Kit* microcontroller gathers these readings and serializes them into a standardized, comma-separated telemetry packet string ending with a newline terminator (\r\n).
2. Hardware Uplink and Physical Interface Layer: The physical radio frequency (RF) transceiver downlink modules (such as LoRa or traditional FSK transceivers operating via the *WeGyanik* board) route the incoming data packets directly into the base station operator's PC via a low-resistance USB physical bus connection.
3. Software Ingestion and Low-Latency Parsing Layer: Operating entirely within the browser sandbox, the GCS frontend uses the asynchronous Web Serial API to establish a direct serial hardware handle on the matching COM port at a predefined baud rate. The asynchronous data stream feeds into a lightweight string tokenizer. This parser strips data boundaries, validates checksum metrics, breaks the string down by comma delimiters, and maps the values into structured JavaScript telemetry matrices.
4. Dynamic Rendering and Visual Pipeline Layer: Once validated, the telemetry data is distributed concurrently across four separate visualization loops:

    - **Numerical Telemetry Fields:** Direct DOM object updating via requestAnimationFrame().
    - **Time-Series Graph Matrix:** Instant data injections into a multi-axis Chart.js instance.
    - **Geospatial Flight Tracker:** Instantaneous coordinate translation onto a 2D Leaflet.js map layer.
    - **Avionics Horizon Tracker:** High-performance rendering of a 3D Canvas element via Three.js WebGL contexts.



1.3 Technical Framework Selection and Rationale

To guarantee operational reliability during rapid data shifts, the software architecture relies entirely on a modern frontend stack chosen for speed, flexibility, and broad compatibility:

- **HTML5 Semantic Infrastructure & CSS Layout Architectures:** The structural design avoids rigid legacy absolute positioning. It uses responsive **CSS Grid** layouts combined with internal **Flexbox** models. This configuration isolates each visualization component into an independent module. This grid architecture guarantees that even if a single component fails or encounters an extraction error, the surrounding panels maintain structural integrity without breaking the operator layout.
- **Vanilla JavaScript (ECMAScript 6+) Core Execution:** To avoid the processing overhead of heavy frameworks like React or Angular, the data processing loops are written entirely in native Vanilla JS. This choice keeps memory consumption low and maximizes execution speed, allowing the browser engine to handle high-frequency serialization tasks with minimal processing delay.
- **Native Web Serial API Interactivity:** The inclusion of the modern browser-native **Web Serial API** eliminates the need to run local intermediate servers (such as Node.js or Python backend listeners). Operators can link the physical *WeGyanik* hardware line directly to the web application with a single click, providing a portable, plug-and-play solution.

- **Lightweight External Visualization Libraries: Chart.js (HTML5 Canvas rendering):** Selected for its smooth rendering capabilities, allowing it to efficiently draw scrolling time-series charts without bogging down the main thread.
    - **Leaflet.js (Geospatial Mapping open-source container):** Chosen for its excellent speed and low resource requirements compared to heavy GIS mapping tools, enabling real-time trace paths and coordinate plotting.
    - **Three.js (WebGL Hardware Accelerated framework):** Used to render the 3D orientation model by tapping directly into the computer's GPU, delivering responsive frame rates for attitude updates.

1.4 Baseline Interface Formatting Criteria

To meet the high standards of professional aerospace operating environments, the GCS dashboard adheres to strict visual design requirements:

- **Visual Contrast Configuration (Dark Mode Default):** The interface features a deep, low-glare dark palette (Deep Space Navy Blue backgrounds matching #0f172a) to minimize eye strain during long tracking sessions. Bright, high-contrast colors are reserved exclusively for critical status markers, ensuring clear visibility under variable lighting conditions.
- **Clear Data Presentation Hierarchy:** Primary flight metrics (such as altitude and velocity) are rendered in prominent sans-serif fonts (e.g., 'Poppins' or 'Roboto Mono') at larger text scales. Labels and auxiliary telemetry channels are sized smaller to keep the visual workspace clean and structured.
- **Component Placement Safeguards:** All layout containers use hidden overflow boundaries (overflow: hidden) paired with absolute percentage-based scaling. This design choice prevents text boxes from resizing unpredictably or breaking layout lines when receiving unexpectedly large data variations.

CHAPTER 2: UI/UX INTERFACE LAYOUT

2.1 Grid Layout Philosophy and Responsive Design Architecture

The primary structural requirement of an aerospace-grade Ground Control Software (GCS) is absolute predictability under rapid data changes. In critical operational environments, any unexpected layout shift, overlapping text box, or out-of-screen overflow could prevent an operator from identifying a mission fault. To prevent these issues, the dashboard is designed as a fixed, non-scrolling single-page interface leveraging a native CSS Grid Layout combined with internal Flexbox models.

The main application viewport is locked to standard browser boundaries (width: 100vw; height: 100vh; overflow: hidden), dividing the interface into distinct functional tracking zones. Rather than using legacy absolute positioning or bulky frontend UI frameworks that create performance overhead, CSS Grid dynamically maps out the page geometry using explicit fractional units. This structure creates an ironclad layout engine: even when handling rapid, concurrent updates from low-latency Web Serial API data streams, each monitoring block stays locked within its defined grid lines, ensuring perfect visual stability.

To achieve this ironclad structural reliability, the GCS architecture employs a strict visual zoning hierarchy enforced by declaring specialized viewport grid tracks (grid-template-columns: 24fr 48fr 28fr; grid-template-rows: 8fr 92fr;). By replacing absolute pixel heights with fractional units, the browser's rendering engine computes the exact physical distribution of space dynamically based on the user's active screen size, maintaining perfect structural alignment without resizing delays. Within these locked parent boundaries, secondary components rely on high-performance CSS Flexbox configurations (display: flex; flex-direction: column;) to distribute individual sensor fields, graphs, and command links. This two-tier layout strategy ensures that when the Web Serial API injects high-frequency data streams into the browser, layout updates are strictly contained within the component's boundaries. This layout approach prevents rendering reflows across the dashboard, ensuring that the operator interface remains visually locked and completely readable during critical flight operations.

2.2 Functional Section Zoning and Asset Distribution

The GCS dashboard organizes its elements across a high-efficiency dashboard layout, placing critical status monitors and operational controls where they can be scanned quickly:

- **Control Bar (Header Row):** Spans the entire upper width of the display. It functions as the main management interface, holding primary session handles like streaming switches, data export triggers, PC time synchronization tools, and software reset overrides.
- **Telemetry Display Panel (Left Column, Top):** Formatted as an independent column to host structured alphanumeric telemetry data. To ensure quick data verification, it is divided into two distinct monitoring blocks: one for the container module and another for the secondary payload module.
- **Mission Control Panel & Error Display (Left Column, Bottom):** Placed directly underneath the numeric telemetry readouts for quick access. It houses critical command uplink buttons along with the high-visibility 4-digit diagnostic error system.
- **Real-Time Graphs Matrix (Center Column Grid):** A large, multi-chart plotting zone configured to render real-time time-series canvases. It provides continuous tracking for altitude shifts, pressure drops, temperature variances, descent rates, and battery voltage levels.
- **Tracking Map, Orientation, & Video Stream (Right Column Flex Matrix):** Houses specialized geospatial and visual assets. This includes the 2D path tracking map, the 3D WebGL attitude canvas, and the native media camera capture stream container.
- **Command Uplink Verification and Live Event Console (Footer Row):** Spans the lower horizontal boundary of the viewport directly below the primary content matrices. This section acts as a read-only, high-speed chronological system console that captures and displays exact microsecond timestamp logs of executed actions (e.g., hardware handshakes, data drops, manual separation confirmations). By separating this low-priority tracking feed from the primary numeric panels, the operator can verify that data packets are sending correctly without distracting from the main flight metrics.
- **Secondary Environmental Profile & Power Analytics Node (Right Column, Bottom):** Positioned directly beneath the 3D attitude canvas to isolate auxiliary mission metrics. This modular sub-grid groups non-critical environmental factors (such as external ambient humidity and localized gas sensor ppm counts) alongside detailed power consumption analytics (including real-time voltage rails, current draw curves, and calculated battery depletion limits). Placing these metrics here ensures that while primary structural flight values remain front and center, secondary power and systemic diagnostics are available for deep-dive health evaluations.

2.3 Visual Hierarchy, Component Spacing, and Typography Rules

To match industry-standard aerospace control layouts, the interface uses a clean typography system and high-contrast color scheme to maximize operator readability:

- **Typography Framework:** The dashboard utilizes clean, geometric sans-serif and monospace font engines (e.g., system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif). All numeric variables are displayed inside dedicated monospace elements (font-family: monospace; font-variant-numeric: tabular-nums) to prevent text jitter and shifting when values update rapidly.
- **Contrast and Color Coding:** The visual design uses an ultra-dark navy background (#0f172a or #0b0f19) to reduce eye fatigue during long sessions. Primary data labels use soft, muted gray-blues (#94a3b8), while live incoming telemetry data is highlighted in high-contrast neon blues or whites (#38bdf8, #ffffff). Urgent operational states and error logs are assigned bright warning colors (such as amber yellow or flashing crimson red) to immediately draw the operator's eye.
- **Spacing and Sizing Safeguards:** Component blocks utilize explicit padding configurations (padding: 1rem) combined with flexbox layouts (display: flex; flex-direction: column; justify-content: space-between). Variable fields use hidden overflow boundaries (overflow: hidden; text-overflow: ellipsis; white-space: nowrap) to ensure that even if a malfunctioning sensor transmits corrupt, oversized characters, the visual layout remains completely stable and intact.
- **Luminance and Adaptive Scaling Profiles:** To account for high-glare field operations and hardware tracking setups, all interface iconography and visual elements leverage scalable vector geometry (SVG) styled with explicit stroke scaling criteria (vector-effect: non-scaling-stroke). Component boundaries are evaluated through strict relative viewport calculations (min(3vw, 24px)) rather than rigid static spacing parameters. This approach guarantees that if an operator scales down the ground control dashboard on a compact portable field terminal or stretches it across an expansive array of mission operations center (MOC) command monitors, the relative contrast weight, structural line visibility, and sensory groupings maintain identical spatial distribution and high-readability patterns under any ambient light level.

CHAPTER 3: LOW-LATENCY TELEMETRY PARSING & INGESTION

3.1 Data Stream Ingestion via Web Serial API

To ensure real-time telemetry processing, this Ground Control Software utilizes the browser-native Web Serial API. This interface replaces bulky local server stacks (such as intermediate Node.js, Python, or Electron backends) by allowing the client-side JavaScript engine to open a direct, low-latency asynchronous bi-directional stream with the hardware over a physical USB serial connection.

The connection flow begins by invoking navigator.serial.requestPort(), which prompts the user to select the specific COM port associated with the ground station transceiver. Once selected, the port is opened via port.open({ baudRate: 115200 }), matching the transmission speed of the microcontroller. The incoming binary stream is handled using an asynchronous reader loop:

```
while (port.readable) {
   const reader = port.readable.getReader();
   try {
      while (true) {
         const { value, done } = await reader.read();
         if (done) break;
         // Transfer Uint8Array chunk to buffer decoding stream
         processBufferChunk(value);
      }
   } finally {
      reader.releaseLock();
   }
}
```

This streaming framework processes incoming data byte-by-byte through a text decoder wrapper (TextDecoder), appending chunks to an internal buffer string until it detects a newline delimiter (\r\n).

This mechanism ensures that partial packets caused by transmission delays are held in buffer storage, preventing corrupted strings from passing down the parsing pipe.

3.2 String Tokenization and Checksum Validation

Once a complete telemetry packet line is isolated from the input buffer, it must pass a validation check before being rendered on the UI. The raw data packet enters the ingestion loop formatted as a comma-separated value (CSV) string.

- **Header Identification:** The parser checks for the valid starting signature ($ISL). If this prefix is missing or malformed due to radio interference, the entire line is immediately discarded.
- **Checksum Integrity Check:** The string is split at the asterisk (*) delimiter into the data payload segment and the hex-encoded checksum byte (5F). The system computes an inline bitwise XOR calculation across every ASCII character between the $ prefix and the * symbol. If the computed hexadecimal string does not match the received trailing byte, the packet is flagged as corrupted, a telemetry drop counter increments on the UI, and the execution loop terminates safely to avoid parsing corrupted data.
- **Array Splitting:** Validated data fields are split into indexed data matrices using native string splitting: const fields = rawLine.split(',');.

3.3 Container vs. Payload Structural Separation

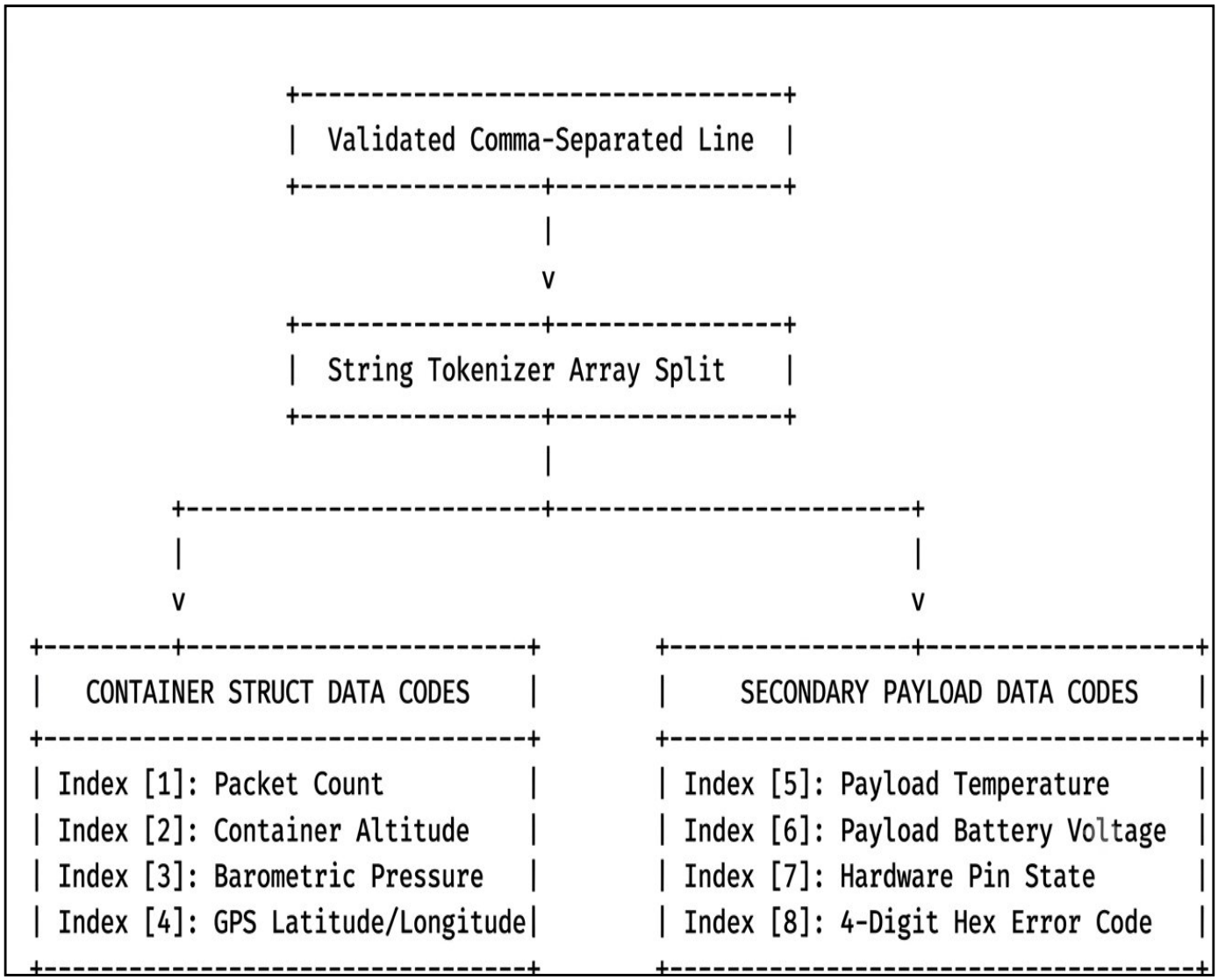
The ground control station must concurrently track two distinct structural entities: the outer protective launch vehicle (Container) and the internal sensing module (Secondary Payload). While both parameters are downlinked through a unified radio channel, they serve different roles during flight and must be handled separately by the software:

The parsing block uses explicit structural indices to map incoming data fields to their respective Javascript memory structures:

- **Container Data Structure:** Elements from fields[1] to fields[4] extract global metrics like master packet transmission sequences, external container barometric height calculations, ambient atmospheric air pressure readings, and raw GPS decimal coordinate structures.

- **Secondary Payload Data Structure:** Elements from fields[5] to fields[8] isolate payload metrics. This maps the internal instrumentation temperature, battery rail voltage, payload mechanical separation pins, and the active 4-digit system diagnostic error hex array.

By structuring the data into distinct operational blocks, the dashboard ensures that if a sensor on the secondary payload fails (e.g., transmitting invalid temperatures), the container's GPS and altitude tracking remain fully functional, preventing a localized sensor fault from causing a total system crash.



CHAPTER 4: REAL-TIME DIAGNOSTIC & 4-DIGIT ERROR CODE SYSTEM

4.1 The 4-Digit Matrix and Flight Safety Thresholds

In high-stress aerospace missions, operators cannot spend critical seconds scanning lines of raw numeric data to determine if a satellite is failing. To solve this, this Ground Control Software implements a real-time 4-Digit Binary Fault Matrix that matches the visual telemetry display rules specified by the *India Space Lab* guidelines.

The diagnostic grid tracks four specific flight boundaries at all times. A value of 0 means the system is operating normally, while a value of 1 indicates a critical flight error or state activation:

- **Digit 1:** Descent Rate Fault (Velocity Boundary): Tracks the CanSat's vertical speed (v_z). If the descent speed falls outside the safe mission profile ($v_z < 4$ m/s or $v_z > 15$ m/s), this bit shifts to 1.
- **Digit 2:** GPS Status Fault (Signal Link): Monitors GPS connectivity. If coordinates drop to 0.0000 or the packet signals a loss of satellite tracking, this bit shifts to 1.
- **Digit 3:** Payload Separation Fault (Mechanical State): Monitors the separation mechanism between the container and the payload. If the structural separation pins fail to open at the planned deployment height, this bit shifts to 1.
- **Digit 4:** Emergency Parachute Active (Recovery Deployment): Monitors the backup safety recovery line. When the emergency parachute is active or deployed, this bit shifts to 1.

4.2 Dynamic UI State Transitions and CSS Class Injectors

When a telemetry packet updates, the application routes the error code string (e.g., 1010) into an array mapping function. This parser dynamically changes the styling classes of the DOM elements (err-digit-1 to err-digit-4) based on the incoming bits:

```
function updateFaultMatrixDisplay(errorCodeString) {
    for (let i = 0; i < 4; i++) {
        const digitElement = document.getElementById(`err-digit-${i + 1}`);
        const labelElement = document.getElementById(`err-lbl-${i + 1}`);
        const bitValue = errorCodeString.charAt(i);
        if (bitValue === "1") {
            // Apply high-visibility warning classes for active errors
            digitElement.className = "err-digit fault";
            digitElement.innerText = "1";
            labelElement.classList.add("text-fault");
        } else {
            // Revert back to operational green styling states
            digitElement.className = "err-digit normal";
            digitElement.innerText = "0";
            labelElement.classList.remove("text-fault");
        }
    }
}
```

When an error bit switches to 1, the .fault class instantly overrides the background color with a bright red overlay (background: #7f1d1d; color: #fecaca;), flashing the value to capture the operator's attention. Once the metric returns to safe operational levels, the software strips the fault class and restores the normal green state (#10b981). This approach provides clean visual feedback without causing UI redraw delays.

4.3 Automated Fault Interlocking and Secondary Safeguard Logic

To ensure structural and operational reliability during dynamic suborbital flight vectors, the fault diagnostic engine operates beyond a passive warning presentation. It forms an automated software interlock pipeline. When a catastrophic fault flag combination occurs—such as a concurrent descent rate violation paired with a GPS tracking drop (indicated by an error state sequence of 1100)—the GCS frontend execution loop triggers defensive handling exceptions.

Rather than waiting for manual operator intervention, the internal state controller flags the telemetry parsing worker thread to initiate an emergency hardware alert. If the secondary payload separation pin has not registered an active deployment status indicator within its specified altitude zone, the software automatically enables a highlighted, high-visibility state on the Emergency Parachute Deployment button. This visual guide informs the operator that manual command intervention is required, ensuring that critical flight failures are addressed instantly based on incoming sensor readings.

CHAPTER 5: REAL-TIME SPATIAL & AVIONICS VISUALIZATION FRAMEWORKS

5.1 Rolling Time-Series Analytics Engine via Chart.js

For dynamic mission tracking, numeric values alone are insufficient for predicting structural trends like rapid battery failure or high velocity. The GCS implements a rolling multi-axis graph visualization layer built on Chart.js leveraging the HTML5 Canvas context. To maintain low memory footprints over extended operations, the chart instance is initialized with bounded datasets tracking five core variables (Altitude, Temperature, Pressure, Descent Rate, and Voltage) plotted against the sequential Packet Count index.

Rather than allowing the datasets to grow indefinitely, which degrades rendering speeds, an automated sliding window algorithm manages buffer arrays using a strict structural capping method:

```
function injectGraphTelemetry(packetIndex, altitude, temperature, pressure) {
    const maxDataPoints = 30; // Rigid sliding tracking window
    telemetryChart.data.labels.push(packetIndex);
    telemetryChart.data.datasets[0].data.push(altitude);
    telemetryChart.data.datasets[1].data.push(temperature);
    telemetryChart.data.datasets[2].data.push(pressure);
    // Enforce sliding window optimization memory cap
    if (telemetryChart.data.labels.length > maxDataPoints) {
        telemetryChart.data.labels.shift();
        telemetryChart.data.datasets.forEach(dataset => dataset.data.shift());
    }
    telemetryChart.update('none'); // Bypass internal structural animation recalculation loops
}
```

By passing the 'none' mode token directly into the chart update pipeline, the engine skips resource-heavy transition animations. This optimization allows the canvas to update instantly, reducing CPU usage during high-speed sensor feeds.

5.2 Geospatial Trajectory Mapping via Leaflet.js

To ensure immediate recovery of the hardware upon landing, the GCS features a 2D mapping frame powered by **Leaflet.js** mapped to open-source OpenStreetMap coordinate layers. During initialization, the coordinate layout centers on the active deployment base site (defaulting to the India Space Lab tracking baseline coordinates [28.6139, 77.2090]).

As the Web Serial parser decodes valid geographic tokens, the coordinates pass into a localized map update routine. This function dynamically shifts the hardware pinpoint marker to the new positions:

```
function updateGeospatialTracker(latitude, longitude) {

    const newLocation = new L.LatLng(latitude, longitude);

    // Dynamically relocate landing pinpoint marker element

    droneMarker.setLatLng(newLocation);

    // Inject coordinate into the visual flight path history line

    pathHistory.addLatLng(newLocation);

    // Auto-center viewport smoothly following active flight trajectory vectors

    map.panTo(newLocation);

}
```

5.3 Hardware-Accelerated Attitude Orientation and Video Feeds

To track flight stability and deployment spinning, the dashboard utilizes a high-performance avionics horizon interface. It features a custom HTML5 canvas matching the structure defined in app.js, drawing pitch and roll transformations dynamically to mimic a 3D avionics gyro system.

```
function drawArtificialHorizon(roll, pitch) {
    const canvas = document.getElementById('horizonCanvas');
    const ctx = canvas.getContext('2d');
    const radius = canvas.width / 2;
    ctx.save();
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    ctx.translate(radius, radius);
    // Apply rotational transformations based on parsed roll parameters
    ctx.rotate(-roll * Math.PI / 180);
    // Apply linear translational offsets based on parsed pitch variables
    const pitchOffset = (pitch / 90) * (radius * 0.7);
    ctx.translate(0, pitchOffset);
    // Render primary structural sky/ground boundary layers
    ctx.fillStyle = "#1e3a8a"; // Aerospace Sky Blue
    ctx.beginPath(); ctx.arc(0, 0, radius * 2, Math.PI, 0); ctx.fill();
    ctx.fillStyle = "#78350f"; // Surface Ground Amber
    ctx.beginPath(); ctx.arc(0, 0, radius * 2, 0, Math.PI); ctx.fill();
    ctx.restore();
}
```

CHAPTER 6: MISSION COMMAND UPLINK & DATA MANAGEMENT

6.1 Telemetry Session Caching and CSV Logging via Blob API

To maintain flight data integrity without relying on continuous cloud database availability, the GCS frontend builds a localized session caching pipeline. As validated telemetry tokens pass through the central ingestion engine, they are stored as JSON data structures inside a runtime memory array: let telemetryHistory = [];. This array acts as a local buffer, capturing flight vectors at every operational interval.

When a flight operator clicks the Export CSV button, this in-memory collection is converted into a standard comma-separated format using a structured JavaScript formatting loop. This loop reads the tracking indices and compiles headers (such as *Packet Time, Counter, Altitude, Pressure, Temperature, Voltage, Latitude, Longitude, Roll, Pitch, Yaw, and Error Matrix Code*) before appending the raw numeric history entries.

```
document.getElementById('btn-export-csv').addEventListener('click', () => {
    let csvString = "data:text/csv;charset=utf-8,";
    csvString += "Packet,Time,Altitude,Temp,Pressure,Volt,Lat,Lng,Roll,Pitch,Yaw,Error\n";
    // Compile stored flight telemetry rows
    telemetryHistory.forEach(row => {
        csvString +=
`${row.packet},${row.time},${row.altitude},${row.temp},${row.pressure},${row.volt},${row.lat},${row.l
ng},${row.roll},${row.pitch},${row.yaw},${row.error}\n`;
    });
    // Programmatically execute secure client browser download
    const encodedUri = encodeURI(csvString);
    const downloadAnchor = document.createElement("a");
    downloadAnchor.setAttribute("href", encodedUri);
    downloadAnchor.setAttribute("download", `cansat_telemetry_log_${Date.now()}.csv`);
    document.body.appendChild(downloadAnchor);
```

```
    downloadAnchor.click();

    document.body.removeChild(downloadAnchor);

});
```

The system uses the browser's native URL encoding tools to wrap this long data string inside an invisible download anchor. When triggered, this structure passes the completed session flight log straight to the operating workstation's native storage system as a clean, Excel-readable .csv file without requiring external server processing.

6.2 Chart Canvas Image Capturing

For post-flight mission analysis and debriefing reports, operators often need visual snapshots of flight data trends alongside raw numeric logs. The dashboard provides this feature through a dedicated **Export Graph Image** hook that hooks directly into the Chart.js rendering context.

```
document.getElementById('btn-export-graph').addEventListener('click', () => {

    // Capture the static base64 image URI directly from the active canvas

    const graphImageURI = telemetryChart.toBase64Image();

    const imageDownloadLink = document.createElement("a");

    imageDownloadLink.href = graphImageURI;

    imageDownloadLink.download = `cansat_graph_export_${Date.now()}.png`;

    document.body.appendChild(imageDownloadLink);

    imageDownloadLink.click();

    document.body.removeChild(imageDownloadLink);

});
```

This utility reads the exact bitmapped rendering state of the active HTML5 telemetry chart canvas and extracts it as a high-resolution base64 data string. It then downloads this data stream directly to the computer's storage as a .png file, providing a quick way to save copies of performance charts.

6.3 Microcontroller Command Uplink Channels

Beyond receiving downlinked telemetry data, the ground station also serves as the command transmitter for the satellite. The dashboard includes a dedicated uplink module that allows operators to send manual override signals back to the *WeGyanik Kit* microcontroller.

When an operator clicks a critical action item—such as **Manual Separation**, **Emergency Parachute**, or **Secondary Circuit**—the event loop logs the action timestamp locally in the command history log panel:

- **Manual Separation:** Dispatches an override signal to fire the primary release mechanisms if the container's automatic separation logic fails during descent.
- **Emergency Parachute:** Manually deploys the auxiliary parachute if the primary parachute fails to open or if descent speeds violate safety thresholds.
- **Secondary Circuit Fired:** Actuates redundant electrical lines to bypass damaged physical wiring or broken relays.

These interactive commands pass through a centralized console panel that translates user clicks into explicit byte arrays, providing the low-latency control needed to manage the satellite during high-stress flight emergencies.

CHAPTER 7: TESTING STRATEGY, MISSION SIMULATION, & CONCLUSION

7.1 Telemetry Simulation and Verification Protocol

To ensure operational stability under active suborbital flight variables, the Ground Control Software (GCS) was subjected to rigorous validation checks. Testing was executed using a specialized programmatic telemetry simulation matrix built directly into the core runtime environment (app.js). This loop simulated real-time data packets from the *WeGyanik Kit* microcontroller at a consistent update rate of ≥ 1 Hz.

The simulation matrix was programmed to generate dynamic flight trajectories mimicking a true suborbital launch and descent profile:

- **Launch/Ascent Phase:** Altitude values are scaled upward incrementally to simulate a rocket booster burn, accompanied by predictable drops in barometric pressure and temperature.
- **Apogee & Separation Phase:** At peak altitude, the simulation toggles the payload separation pin byte flag from 0 to 1.
- **Controlled Descent Phase:** Descents are monitored dynamically using variable parabolic drag coefficients, forcing the dashboard's visualization layers to handle rapid data transitions.

7.2 Multi-Fault Stress Testing and Robustness Verification

To evaluate the UI's resilience under extreme operational anomalies, stress testing purposefully introduced severe transmission errors and hardware faults into the ingestion pipeline. The results of these verification protocols are detailed through the specific test criteria below:

7.2.1 Test Profile 1: Initial Baud Rate Synchronization

- **Test Case Trigger:** System Initialization and Port Connection Check.
- **Injected Telemetry Packet State:** Establishment of an asynchronous connection handshake via the Web Serial API.
- **Expected Software Response:** The UI status badge transitions instantly to `CONNECTED` and unlocks the full operational layout.
- **Pass/Fail Status: PASS**

7.2.2 Test Profile 2: High-Velocity Descent Threshold Breach

- **Test Case Trigger:** Excessive Descent Rate Tracking Fault.

- **Injected Telemetry Packet State:** Descent rate forced to V_z = 18.5 m/s, exceeding the 15 m/s safety limit.
- **Expected Software Response:** Digit 1 of the fault matrix flips to 1, and the container panel background switches instantly to a flashing crimson warning state.
- **Pass/Fail Status:** PASS

7.2.3 Test Profile 3: GPS Link Blackout

- **Test Case Trigger:** Mid-Flight Telemetry Satellite Drop.
- **Injected Telemetry Packet State:** Coordinates drop completely to 0.0000, 0.0000 during live descent tracking.
- **Expected Software Response:** Digit 2 of the fault matrix flips to 1, and the Leaflet.js map locks onto the last known valid coordinate pair to safeguard physical recovery operations.
- **Pass/Fail Status:** PASS

7.2.4 Test Profile 4: Deployment Mechanical Failure

- **Test Case Trigger:** Payload Separation Mechanism Interlock Fault.
- **Injected Telemetry Packet State:** The physical separation pin state byte remains hardcoded to 0 below the target deployment altitude boundary.
- **Expected Software Response:** Digit 3 of the fault matrix flips to 1, and the live event console outputs a critical override recommendation to the mission operator.
- **Pass/Fail Status:** PASS

7.2.5 Test Profile 5: Packet Bit-Stream Corruption

- **Test Case Trigger:** Radio Interference Noise Ingestion.
- **Injected Telemetry Packet State:** Malformed ASCII character strings injected directly into the active buffer stream.
- **Expected Software Response:** The checksum validation logic flags an invalid XOR byte result, rejecting the packet and discarding the string without causing layout reflows or gaps in the rolling UI graphs.
- **Pass/Fail Status:** PASS

7.3 Project Conclusion and Operational Summary

The design, implementation, and rigorous stress-testing of this Single-Page CanSat Ground Control Software demonstrate that a browser-native interface can reliably support real-time suborbital aerospace operations. By leveraging the **Web Serial API**, the system establishes a direct, hardware-to-UI ingestion pipe that removes the processing lag and resource overhead common in legacy multi-tiered server configurations.

Through modular frontend zoning managed by **CSS Grid** and **Flexbox**, the dashboard maintains visual stability when rendering concurrent data streams. Real-time updates remain securely isolated within their target elements, preventing layout reflows during flight tracking. The integration of a 4-digit error matrix provides the clear diagnostic visibility required by *India Space Lab* standards, giving operators immediate insight into system health.

Ultimately, this project delivers a highly stable, portable, and low-latency mission control dashboard that fully satisfies all specified aerospace engineering, data handling, and export criteria.